

AUDITORÍA TÉCNICA Y PEDAGÓGICA

Electude Simulator Challenge

Análisis funcional completo y hoja de ruta para el desarrollo de un simulador propio

Preparado para: Academias Ethermic

Fuente analizada: simulator.electude.com

Julio 2026

Índice

1. Resumen ejecutivo
2. ¿Qué es el Electude Simulator Challenge?
3. Auditoría funcional detallada del sistema
4. Mecánica de evaluación y puntuación (gamificación)
5. Arquitectura técnica inferida
6. Fundamento pedagógico
7. Hoja de ruta para construir un sistema propio
8. Stack tecnológico recomendado (2D vs. 3D)
9. Ideas de mejora e innovación para Academias Ethermic
10. Consideraciones legales y de propiedad intelectual
11. Plan de fases sugerido

1. Resumen ejecutivo

El Electude Simulator Challenge es una herramienta de e-learning para formación automotriz, desarrollada por la empresa neerlandesa Electude (fundada en 1990). Simula un sistema completo de gestión de motor (Engine Management System) con sensores, actuadores, módulos de control (ECU), red CAN y herramientas de diagnóstico reales (osciloscopio, multímetro, breakout box, escáner OBD). El estudiante recibe una orden de trabajo con una falla, debe diagnosticarla usando instrumentos virtuales, reemplazar componentes o cables, y devolver el vehículo, obteniendo una calificación basada en velocidad, costo y orden del proceso.

Esta auditoría documenta en detalle cada mecánica del sistema (para que sirva de especificación funcional) y propone cómo Academias Ethermic puede construir una versión propia — igual o superior — apoyándose en motores 3D web (Three.js/Babylon.js/Unity), un motor de simulación de fallas basado en circuitos, y un sistema de autoría para que los instructores creen sus propios ejercicios. No se trata de copiar assets ni código de Electude (esto violaría derechos de autor), sino de replicar el concepto funcional y pedagógico con tecnología y diseño propios.

2. ¿Qué es el Electude Simulator Challenge?

Es la versión gratuita/demo del simulador comercial de Electude, usado como gancho de marketing y como reto de puntuación pública ("high scores"). La versión completa ("Connect Engine Management") se vende integrada dentro de su LMS a escuelas técnicas y programas vocacionales (ASE/NATEF en EE.UU.), y existe también en formato de hardware físico (ConsuLab) con el mismo panel didáctico.

2.1 Dos modos de acceso

- Modo "Challenge" (con registro): el estudiante inicia sesión, recibe fallas en tres niveles de dificultad crecientes y compite en una tabla pública de puntajes.
- Modo "Simulador libre" (sin registro): acceso directo a la simulación completa del motor para explorar libremente sensores, actuadores y herramientas, sin objetivo de puntuación.

2.2 Público objetivo y contexto de uso

- Estudiantes de mecánica/electricidad automotriz en escuelas técnicas.
- Instructores, que en la versión completa pueden definir y asignar fallas y personalizar las órdenes de trabajo.
- Autodidactas o técnicos que practican diagnóstico antes de tocar un vehículo real.

3. Auditoría funcional detallada del sistema

A continuación se describe, componente por componente, cada elemento observado/documentado del simulador. Esta sección funciona como especificación funcional base para una réplica.

3.1 Orden de trabajo (Work Order)

Es el punto de entrada de cada ejercicio. Contiene la descripción del cliente sobre la falla (síntoma, no diagnóstico). El estudiante debe:

1. Leer la descripción del problema.
2. Diagnosticar usando las herramientas disponibles.
3. Reparar (reemplazar componente o cable defectuoso).
4. Completar por escrito el diagnóstico y la solución aplicada.
5. Marcar un checklist de cierre (ver sección 4).
6. Pulsar "Devolver vehículo al cliente", lo que dispara la calificación final.

3.2 Panel de instrumentos (Instrument Panel / Dashboard)

Permite arrancar el vehículo y accionar el acelerador. Muestra revoluciones y temperatura del motor en tiempo real (si el panel está conectado a la ECU) y tres testigos de advertencia (luces de fallo).

3.3 Osciloscopio de 2 canales

Instrumento fiel a uno real, con las siguientes funciones documentadas:

- Dos sondas (canal 1 rojo, canal 2 azul) que se conectan a cualquier punto de "back-probing" del vehículo, más pinza de tierra a batería.
- Encendido/apagado, control de zoom vertical (V/div) independiente por canal, y horizontal (s/div) compartido.
- Botón "Auto" que autoconfigura escala y disparo (autoset, como en un osciloscopio real Fluke/Snap-on).
- Cuatro menús: Canal 1 (señal + supresión de ruido), Trigger (canal y pendiente de disparo), Canal 2, y Memoria (5 capturas guardables).
- Función Hold/Run para congelar la señal en vivo.

3.4 Multímetro

Mide voltaje y resistencia entre dos puntos (sondas roja y negra), pudiendo conectarse a conectores de componentes o al breakout box. Los puntos disponibles se resaltan en celeste al arrastrar la sonda. Selector DC/AC y perilla de sensibilidad/función.

3.5 Sistema de diagnóstico (Escáner / Diagnostic Tester)

Emula un escáner profesional tipo OBD-II. Flujo de uso:

7. Identificación del vehículo (obligatorio antes de cualquier función).
8. Menú principal con tres rutas: "Todos los códigos de falla", "Sistemas" y "Módulos de control".
9. Para un sistema o módulo específico: Información (diagrama de bloques CAN y diagrama eléctrico interactivo, clicable componente por componente), Códigos de falla, Datos en vivo (live data), Prueba de actuadores (actuador test / bidireccional), Configuración y Programación (reflasheo del firmware).

3.6 Breakout box (caja de interrupción)

Se ubica físicamente junto a la ECU (sobre el amortiguador en la simulación). Para usarla hay que desconectar primero el conector original de la ECU; luego puede conectarse al arnés, a la ECU, o a ambos simultáneamente, habilitando mediciones "en vivo" sin dañar los pines del conector original — tal como en un taller real.

3.7 Manómetro de presión de combustible

Mide la presión relativa en el riel de inyectores; se conecta de forma inmediata (sin pasos adicionales), simulando un manómetro mecánico de taller.

3.8 Arrancador auxiliar (Jump Starter)

Se conecta a la batería (si las pinzas están conectadas) y se ubica junto al filtro de aire, cerca de la batería.

3.9 Diagrama de cableado (Wiring Diagram)

Muestra el esquema eléctrico completo del sistema de gestión del motor. Permite localizar y reemplazar un cable roto directamente desde el diagrama. El arnés está dividido en sectores; el estudiante debe medir para determinar en qué sector exacto está la falla antes de reemplazar el cable correcto (evita "adivinar").

3.10 Barra de componentes y reemplazo físico

- Los componentes desconectados pueden arrastrarse a una barra inferior (que se expande al pasar el mouse o al tocarla en pantallas táctiles).
- Un componente en la barra puede reemplazarse con un clic; el nuevo componente se resalta en amarillo y su precio se agrega automáticamente a la factura.
- Ciertos componentes (inyectores, bobinas de encendido, bujías, relé de 4 polos) permiten "intercambio de posición": se saca uno a la barra, se mueve el otro a su lugar, y luego se coloca el primero en la posición vacía — mecánica clásica para diagnosticar fallas por sustitución cruzada.
- El reemplazo/reinstalación solo es posible con zoom suficiente sobre el componente (evita errores por imprecisión de clic).

3.11 Factura (Invoice) en tiempo real

Se actualiza automáticamente y registra: tiempo de diagnóstico, componentes reemplazados con su precio de catálogo, y el tiempo adicional que tomaría desmontar/montar cada pieza en un vehículo real (aunque en la simulación el reemplazo sea instantáneo). Incluye una lista de precios de referencia al pie.

4. Mecánica de evaluación y puntuación (gamificación)

Este es el corazón del diseño de "reto" (Challenge) y lo que lo hace adictivo y competitivo. Reglas documentadas:

4.1 Regla de intentos

- Primer intento exitoso: hasta 100 puntos.
- Si no se resuelve, se puede reintentar, pero el puntaje máximo posible se reduce a la mitad en cada intento (100 → 50 → 25 → 12.5).
- Máximo 4 intentos por ejercicio.

4.2 Distribución del puntaje (dentro del máximo posible del intento)

Componente del puntaje	Peso	Criterio
Eficiencia de reparación	50%	Relación entre el costo real generado y el costo mínimo necesario. Ej.: gastar el doble de lo necesario en piezas/tiempo → solo 25 de 50 puntos posibles.
Orden y buenas prácticas ("trabajo prolijo")	50%	Checklist de 4 puntos: (1) ¿todos los conectores están correctamente conectados?, (2) ¿no quedan códigos de falla activos?, (3) ¿se guardaron todas las herramientas?, (4) ¿fusibles bien, batería cargada y bujías limpias?

Ejemplo documentado: olvidar borrar los códigos de falla o no guardar el multímetro reduce el segundo bloque a 25 de 50 puntos, aunque la reparación en sí haya sido correcta y eficiente.

4.3 Niveles de dificultad

El modo Challenge presenta tres niveles progresivos. Electude no publica el detalle interno de cada nivel, pero por el diseño del sistema y la documentación disponible, el criterio de progresión combina:

- Cantidad de sistemas involucrados en la falla (un solo sensor vs. interacción entre módulos vía CAN).
- Ambigüedad del síntoma reportado por el "cliente" (más genérico = más difícil).
- Necesidad de usar instrumentos más avanzados (multímetro → osciloscopio → escáner con diagrama CAN).
- Reducción de pistas / mayor cantidad de componentes candidatos a estar fallados.

4.4 Catálogo de fallas

Según material corporativo de Electude, el motor de fallas del producto completo permite a un instructor elegir entre cerca de 60 fallas distintas y personalizar la descripción del cliente en la orden de trabajo — es decir, la falla y su "envoltorio narrativo" están desacoplados, lo cual es clave para el diseño de un sistema propio (ver sección 7).

4.5 Tabla de puntajes (Leaderboard)

Pública, global, ordenada por puntaje descendente, con nombre de usuario y (opcionalmente) escuela/organización. Es un mecanismo simple pero muy efectivo de gamificación social: compite el estudiante individual contra técnicos de todo el mundo.

5. Arquitectura técnica inferida

Electude no publica su arquitectura interna; lo siguiente es una inferencia razonable basada en el comportamiento observable, la fecha de lanzamiento (versión "cloud-based" en 2014) y las capacidades descritas (multi-idioma, multi-dispositivo, funciona en cualquier navegador moderno sin plugins):

Capa	Función	Tecnología probable
Motor gráfico/interacción	Renderizado 2D del vehículo, componentes y herramientas; drag & drop; zoom	Canvas/WebGL 2D vectorial (reemplazó una versión previa en Flash de 2014)
Motor de simulación	Modelo lógico de circuito: continuidad eléctrica, valores de sensores, lógica de la ECU, propagación de fallas	Motor de reglas / grafo de circuito en JavaScript, ejecutado en el cliente
Configurador de fallas	Define qué componente/cable está averiado y qué síntoma genera	Motor de datos parametrizable (backend), separado del motor gráfico
Backend / cuentas	Login, progreso, puntajes, tabla de posiciones	API REST + base de datos relacional
Multilenguaje	Más de 30 idiomas	Archivos de recursos de texto (i18n) independientes del motor

El punto clave para replicarlo es que el "motor de simulación" (el cerebro que decide qué mide cada instrumento según qué está roto) está desacoplado del "motor gráfico" (lo que se ve y se arrastra) y del "catálogo de fallas" (los datos). Esa separación en tres capas es exactamente lo que se recomienda en la sección 7.

6. Fundamento pedagógico

El diseño responde a un modelo de aprendizaje basado en problemas (Problem-Based Learning) y "aprender haciendo", replicando el ciclo real de un taller: síntoma → hipótesis → medición → confirmación → reparación → verificación → cierre administrativo. Puntos pedagógicos relevantes:

- Causa y efecto real: al cambiar un valor de sensor, la ECU reacciona sobre los actuadores, permitiendo entender el sistema como un todo y no memorizar solo "síntoma = pieza".
- Entorno seguro y de bajo costo: se puede "romper" el motor virtual sin riesgo ni gasto de piezas reales.
- Evaluación por proceso, no solo por resultado: el sistema de puntos castiga tanto el exceso de gasto como el desorden, formando hábitos profesionales (borrar códigos, guardar herramientas, no reemplazar de más).
- Progresión de dificultad estructurada, que evita frustración temprana y sostiene motivación (curva de dificultad tipo videojuego).

- Competencia social sana mediante el leaderboard global, que motiva la repetición voluntaria (una de las mejores señales de que la gamificación funciona).

7. Hoja de ruta para construir un sistema propio (Academias Ethermic)

La recomendación no es clonar literalmente el simulador de motor de Electude (asunto de sistema eléctrico automotriz, con sus assets y textos), sino tomar su arquitectura conceptual de 3 capas y aplicarla al dominio que ustedes elijan: puede ser el mismo motor de combustión, un sistema eléctrico de vehículo, una instalación domiciliaria, un tablero industrial, etc. La misma "máquina de simulación de fallas" sirve para cualquier sistema técnico basado en circuitos, sensores y actuadores.

7.1 Las tres capas a construir

Capa 1 — Modelo del sistema (el "cerebro")

Un grafo de nodos y conexiones que represente el circuito real: componentes (con sus propiedades: resistencia, tipo de señal, rango de voltaje), cables/conectores, y reglas de comportamiento (p. ej. "si el sensor de temperatura da una lectura fuera de rango, la ECU activa el testigo X y ajusta el actuador Y"). Cada "falla" es simplemente una alteración de una propiedad de un nodo o arista del grafo (cable cortado = resistencia infinita; sensor fallado = valor fijo o ruidoso; conector flojo = intermitencia). Esta capa se puede programar en JavaScript/TypeScript puro, sin gráficos.

Capa 2 — Instrumentos virtuales

Cada herramienta (multímetro, osciloscopio, escáner) es simplemente un "lector" que consulta la Capa 1 en el punto donde el usuario coloca la sonda y dibuja el resultado (aguja, forma de onda, código de falla). Si el modelo de la Capa 1 está bien hecho, agregar una herramienta nueva no requiere tocar la lógica de fallas.

Capa 3 — Presentación (2D o 3D) e interacción

La capa visual: el vehículo o tablero, el arrastrar/soltar, el zoom, las animaciones. Es la más costosa de producir pero la más reemplazable/mejorable con el tiempo sin afectar la lógica pedagógica.

Capa adicional — Motor de autoría (para instructores)

Un panel de administración donde el instructor arma "casos": elige qué nodo(s) del grafo se alteran, redacta la descripción del cliente para la orden de trabajo, define el precio de las piezas involucradas y el nivel de dificultad. Esto es lo que Electude describe como poder elegir entre ~60 fallas y personalizar el enunciado — es una función de altísimo valor para una academia, porque permite crecer el banco de ejercicios sin programar.

7.2 Módulos de gamificación a implementar

- Sistema de intentos con puntaje decreciente (100/50/25/12.5), igual o mejorado.
- Checklist de cierre configurable por ejercicio (parametrizable, no fijo).
- Factura / bitácora automática de costos y tiempos.
- Leaderboard global y por cohorte/clase (para competencia interna entre secciones de Academias Ethermic).
- Niveles de dificultad con desbloqueo progresivo.

- Panel docente con analítica: qué fallan más los estudiantes, tiempo promedio, reincidencia de errores — esto Electude no lo expone públicamente, y sería una mejora diferencial.

8. Stack tecnológico recomendado (2D vs. 3D)

Ambos caminos son viables; la decisión depende del presupuesto, el tiempo y qué tan "espectacular" quieren que se vea frente a estudiantes.

8.1 Opción A — 2D vectorial interactivo (más rápido y barato, como el original)

- Frontend: React + Konva.js o PixiJS (renderizado 2D de alto rendimiento con drag & drop, zoom, capas).
- Ilustraciones: componentes dibujados en Figma/Illustrator o generados como SVG, exportados como sprites.
- Ventaja: desarrollo más rápido, funciona bien en cualquier dispositivo (incluso gama baja), más fácil de mantener.
- Ideal si el objetivo principal es la simulación funcional del circuito y el diagnóstico, no el "efecto wow" visual.

8.2 Opción B — Modelado y simulación 3D (más impactante, mayor inversión)

- Motor: Three.js o Babylon.js para 3D en el navegador (sin instalar nada, corre en Chrome/Edge/Safari).
- Alternativa con más herramientas de editor visual: Unity (exportado a WebGL) o Unreal Engine (más pesado, menos apto para web), o PlayCanvas (motor 3D basado en navegador, con editor colaborativo en la nube, muy usado para configuradores de producto y simuladores educativos).
- Modelado 3D: Blender (gratuito, muy potente) para crear el motor, componentes, cables, y el chasis; exportar en formato glTF/GLB, el estándar abierto más eficiente para 3D en web.
- Bancos de modelos: se pueden comprar/adaptar modelos base de motores y sistemas eléctricos automotrices en mercados como Sketchfab, CGTrader o TurboSquid, para acelerar el desarrollo en vez de modelar todo desde cero.
- Físicas/animación de piezas: usar animaciones esqueléticas o morph targets para simular apertura de tapas, extracción de componentes, giro de tornillos, etc.
- Ventaja: mucho más atractivo, permite "caminar alrededor" del motor o vehículo, mejor para marketing y diferenciación frente a la competencia.
- Desventaja: mayor costo de producción (artistas 3D), mayor exigencia de hardware/navegador del estudiante, tiempos de carga más largos si no se optimiza bien (LOD, compresión Draco/KTX2).

8.3 Recomendación híbrida (sugerida)

Construir primero la Capa 1 (motor de simulación de fallas) y la Capa 2 (instrumentos) de forma agnóstica a la presentación. Lanzar una primera versión con Opción A (2D) para validar la pedagogía y el motor de autoría rápidamente con estudiantes reales. En una segunda fase, migrar solo la Capa 3 a 3D (Babylon.js/PlayCanvas), reutilizando toda la lógica ya probada. Así se evita el riesgo de gastar el presupuesto grande en arte 3D antes de validar que el motor de fallas y el sistema de puntuación funcionan pedagógicamente.

8.4 Backend y datos

- Autenticación y perfiles de estudiante/instructor: Supabase o Firebase (rápido de implementar, con roles).
- Base de datos: PostgreSQL (vía Supabase) para catálogo de componentes, fallas, precios, intentos, puntajes e historial — coincide con lo que ya usan en el proyecto ZR Help, lo que permite reutilizar conocimiento del equipo.
- Analítica de aprendizaje: registrar cada acción del estudiante (qué midió, en qué orden, cuánto tardó) para un futuro panel de "mapa de calor de errores comunes", muy valioso pedagógicamente y algo que Electude no muestra públicamente.

9. Ideas de mejora e innovación frente al original

- Multi-dominio: no limitarse a motor de gestión electrónica; ofrecer módulos de sistema eléctrico general, frenos ABS, climatización, vehículo eléctrico/híbrido — ampliando el catálogo más allá de lo que Electude cubre en su versión gratuita.
- Asistente con IA integrado: un "copiloto" conversacional (usando un modelo de lenguaje) que dé pistas progresivas si el estudiante lleva mucho tiempo sin avanzar, sin regalar la respuesta — algo que ningún simulador de este tipo ofrece hoy.
- Modo colaborativo: dos estudiantes resolviendo el mismo caso en tiempo real (uno mide, otro decide), fomentando trabajo en equipo tipo taller real.
- Realidad aumentada opcional: usar el modelo 3D ya construido para verlo superpuesto en el aula vía tablet/celular (WebXR), una capa adicional que Electude no ofrece en su versión web.
- Certificación con evidencia: generar automáticamente un reporte/PDF del desempeño del estudiante en cada caso, útil como evidencia de competencia para certificaciones internas de Academias Ethermic.
- Editor de casos accesible para docentes no técnicos: que cualquier instructor arme un ejercicio nuevo en minutos sin programar, ampliando mucho más rápido el banco de ejercicios que el modelo de "~60 fallas" fijas de Electude.

10. Consideraciones legales y de propiedad intelectual

- No se debe copiar código, arte, textos ni la marca de Electude; esta auditoría describe el comportamiento funcional observable (lo cual es lícito de analizar y usar como inspiración conceptual), no el código fuente ni los activos gráficos.
- Los conceptos de "osciloscopio virtual", "orden de trabajo", "puntaje decreciente por intento" son mecánicas educativas genéricas usadas en múltiples simuladores de la industria (ConsuLab, Snap-on, Autodata) y no son exclusivas de Electude.
- Si se contratan modelos 3D de bancos de assets (Sketchfab/TurboSquid/CGTrader), verificar la licencia de uso comercial/educativo de cada modelo antes de integrarlo.
- Recomendable diseñar una identidad visual, nomenclatura de menús y flujo de pantallas propios de Academias Ethermic, para que el producto final sea reconocible como marca propia y no una réplica visual del original.

11. Plan de fases sugerido

Fase	Duración estimada	Entregable
1. Diseño pedagógico y de producto	3-4 semanas	Documento de requisitos, lista de fallas/casos iniciales, diseño de UX de pantallas clave
2. Motor de simulación (Capa 1 y 2)	6-8 semanas	Motor de circuito funcional en JS/TS con multímetro y osciloscopio básicos, sin gráficos finales
3. Prototipo visual 2D jugable	6-8 semanas	Primer caso completo jugable de principio a fin, con orden de trabajo, factura y puntaje
4. Motor de autoría para instructores	4-6 semanas	Panel donde un docente crea un caso

Fase	Duración estimada	Entregable
		nuevo sin programar
5. Piloto con estudiantes reales	2-4 semanas	Métricas de uso, ajustes de dificultad y de puntuación
6. Escalado a 3D (opcional)	8-12 semanas	Migración de la Capa 3 a Babylon.js/PlayCanvas con modelos 3D
7. Analítica e IA de pistas	4-6 semanas	Panel docente con estadísticas y asistente de pistas

Cierre

Este documento resume de forma exhaustiva el comportamiento funcional del Electude Simulator Challenge tal como se observa públicamente, y traduce ese comportamiento en una arquitectura de tres capas (modelo de circuito, instrumentos, presentación) más un motor de autoría, que Academias Ethermic puede construir con tecnología propia — empezando en 2D para validar rápido, y escalando a 3D cuando el motor pedagógico ya esté probado. Quedo disponible para profundizar en cualquier sección: por ejemplo, diseñar el esquema de base de datos del catálogo de fallas, prototipar el motor de circuito en código, o evaluar en detalle Babylon.js vs. PlayCanvas vs. Unity WebGL para el caso específico de ustedes.